

open

דוגמה לקריאת המערכת `open()` בשפת C, המשלבת את כל שלושת הארגומנטים האפשריים (נתיב, דגלים והרשאות יצירה), כולל ניתוח מעמיק של מה שקורה מאחורי הקלעים בקרנל (Linux Kernel 6.14).

קוד לדוגמה במרחב המשתמש (User Space)

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd;

    fd = open("/home/Itzhak/lecture01", O_RDWR | O_CREAT, 0644);

    if (fd == -1) {
        perror("Error opening/creating file");
        return 1;
    }

    close(fd);
    return 0;
}
```

ניתוח מפורט של ארגומנטי הפונקציה

החתימה המלאה של קריאת המערכת כפי שהיא מוגדרת בליבת לינוקס מקבלת שלושה ארגומנטים עיקריים:

```
int open(const char *pathname, int flags, mode_t mode);
```

- א. הארגומנט הראשון: `pathname` ("home/Itzhak/lecture01")
 - משמעות: מחרוזת תווים המצביעה על הנתיב שבו ממוקם הקובץ (או שבו ייווצר).
- פענוח בקרנל: מכיוון שהמחרוזת מתחילה בתו '/', הקרנל מזהה מיד שמדובר בנתיב אבסולוטי (Absolute Path). בתהליך ה-Path Resolution, הקרנל מתעלם מתיקיית העבודה הנוכחית (CWD) של התהליך ומתחיל את הסריקה ישירות מתיקיית השורש הגלובלית (root dentry) של מערכת הקבצים.

ב. הארגומנט השני: flags (O_RDWR | O_CREAT) :
זהו משתנה ביטים (Bitmask) המשיל מספר הגדרות יחד באמצעות פעולת Bitwise OR (|) :

1. O_RDWR (Read-Write) : מורה לקרנל לפתוח את הקובץ במצב המאפשר גם קריאה (read()) וגם כתיבה (write()).

2. O_CREAT (Create if not exists) : מורה לקרנל שאם הקובץ
home/Itzhak/lecture01/ אינו קיים פיזית במערכת הקבצים, יש ליצור אותו באופן
אוטומטי (לייצר לו Inode חדש ולהוסיף רשומה לתיקיית היעד). אם הקובץ כבר קיים,
הדגל הזה מתעלם באדיבות והקובץ נפתח רגיל.

ג. הארגומנט השלישי: mode (0644)

- משמעות: מוגדר אך ורק כאשר משתמשים בדגל O_CREAT ומציין את הרשאות האבטחה הראשוניות של הקובץ החדש במקרה והוא נוצר ישירות כעת.

- פענוח המספר 0644 (אוקטלי):

- הספרה הראשונה (6) מעניקה למשתמש הבעלים (Owner) הרשאות קריאה וכתיבה (read + write).
- הספרה השנייה (4) מעניקה לקבוצה (Group) הרשאת קריאה בלבד (read).
- הספרה השלישית (4) מעניקה לכלל המשתמשים האחרים (Others) הרשאת קריאה בלבד.
- (הערה: ערך זה כפוף תמיד לסינון מסכת ההרשאות של התהליך – ה-umask).

מה קורה בקרנל (Linux Kernel 6.14) מרגע ביצוע הפקודה?

כאשר הפקודה מתבצעת, מתרחש רצף פעולות ארכיטקטוני ברמת הליבה:

1. **מעבר מ-User ל-Kernel Mode:** קריאת ה-open מפעילה פקודת אסמבלי (syscall), והמעבד עובר מ-Ring 3 ל-Ring 0. בגרעין מודרני כמו 6.14, הקריאה מתועלת לפונקציית הליבה הראשית sys_openat (כאשר הפרמטר הראשון המרומוז הוא AT_FDCWD, אך מתעלמים ממנו כי הנתיב אבסולוטי).

2. **העתקת הנתיב ממרחב המשתמש:** הקרנל מעתיק בבטחה את מחרוזת הנתיב מזיכרון המשתמש אל זיכרון הקרנל באמצעות פונקציות העתקה ייעודיות (strncpy_from_user).

3. **Path Resolution (איתור או יצירת Inode):**

- הקרנל סורק את עץ הספריות החל מהשורש (/), עובר דרך התיקיות home ואז .Itzhak.

- הוא מגיע לשם הסופי lecture01.

- אם הקובץ קיים, הקרנל מאתר את ה-Inode שלו בטבלת ה-Inodes.

- אם הקובץ אינו קיים (ובו בזמן צוין הדגל O_CREAT), הקרנל מקצה Inode חדש, רושם אותו ב-Inode Bitmap, יוצר Directory Entry חדש תחת התיקייה .Itzhak, ומחיל את ההרשאות שהועברו ב-mode (בכפוף ל-umask) על שדה ה-i_mode של ה-Inode החדש.

4. **יצירת אובייקט קובץ פתוח (struct file):** הקרנל מקצה בזיכרון מבנה נתונים דינמי בשם struct file, השומר את מצב הפתיחה הנוכחי של התהליך (כגון File Offset שמתחיל כרגע ב-0, והדגלים שנבחרו כמו O_RDWR).

5. **הקצאת File Descriptor (תאור קובץ - FD):**

- הקרנל סורק את טבלת תיאורי הקבצים של התהליך השמורה ב-files-<current (מבנה struct files_struct).

- הוא מוצא את המקום הפנוי הראשון בטבלה (למשל, אינדקס מספר 3).

- הוא רושם בתוך תא זה מצביע אל אובייקט ה-struct file שיצר.

6. **חזרה למרחב המשתמש:** הקרנל מחזיר את המספר השלם החיובי 3 אל פונקציית ה-open במרחב המשתמש. מספר זה (fd) הוא המפתח שבו ישתמש מעתה התהליך בכל פעם שירצה לקרוא (read) או לכתוב (write) לקובץ.